Lezione Python: Cinematica inversa

Fondamenti di Robotica

Docenti: Loredana Zollo
Clemente Lauretti

Tutor: **Rita Molle**
email: rita.molle@unicampus.it

CAMPUS BIO-MEDICO UNIVERSITY OF ROME
Via Álvaro del Portillo, 21 - 00128 Rome - Italy
www.unicampus.it

CREO LAB

UNIVERSITA' CAMPUS BIO-MEDICO DI ROMA

Cinematica inversa

Determinare le variabili di giunto q date la posizione e l'orientamento della terna utensile rispetto alla terna di base

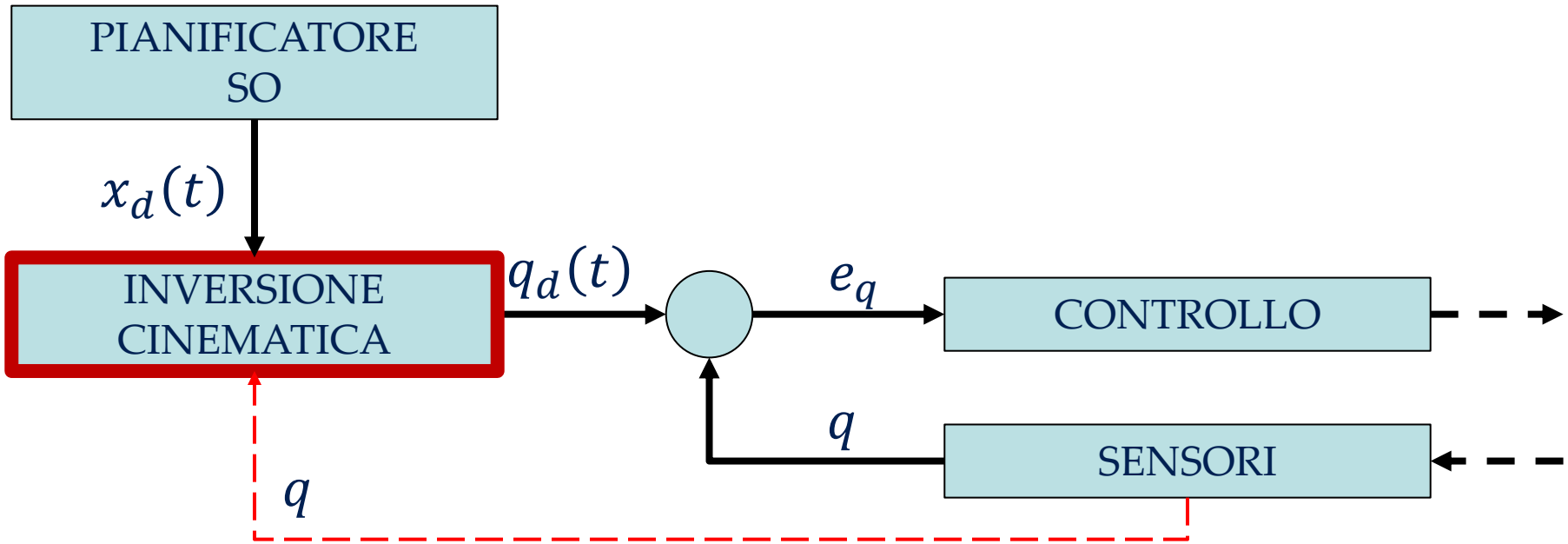
T → q
x → q

La posizione e l'orientamento della terna utensile possono essere date o con la matrice di trasformazione o con le coordinate nello spazio operativo.

- Il problema può non ammettere soluzione (se posizione ed orientamento non appartengono allo spazio di lavoro del manipolatore)
- La soluzione analitica (in forma chiusa) può non esistere, nel qual caso si ricorre a tecniche numeriche
- Possono esserci soluzioni multiple
- Possono esserci infinite soluzioni

In generale la soluzione si ricava senza un procedimento sistematico, ma sulla base di intuizione nella manipolazione delle equazioni.

Algoritmi di inversione cinematica



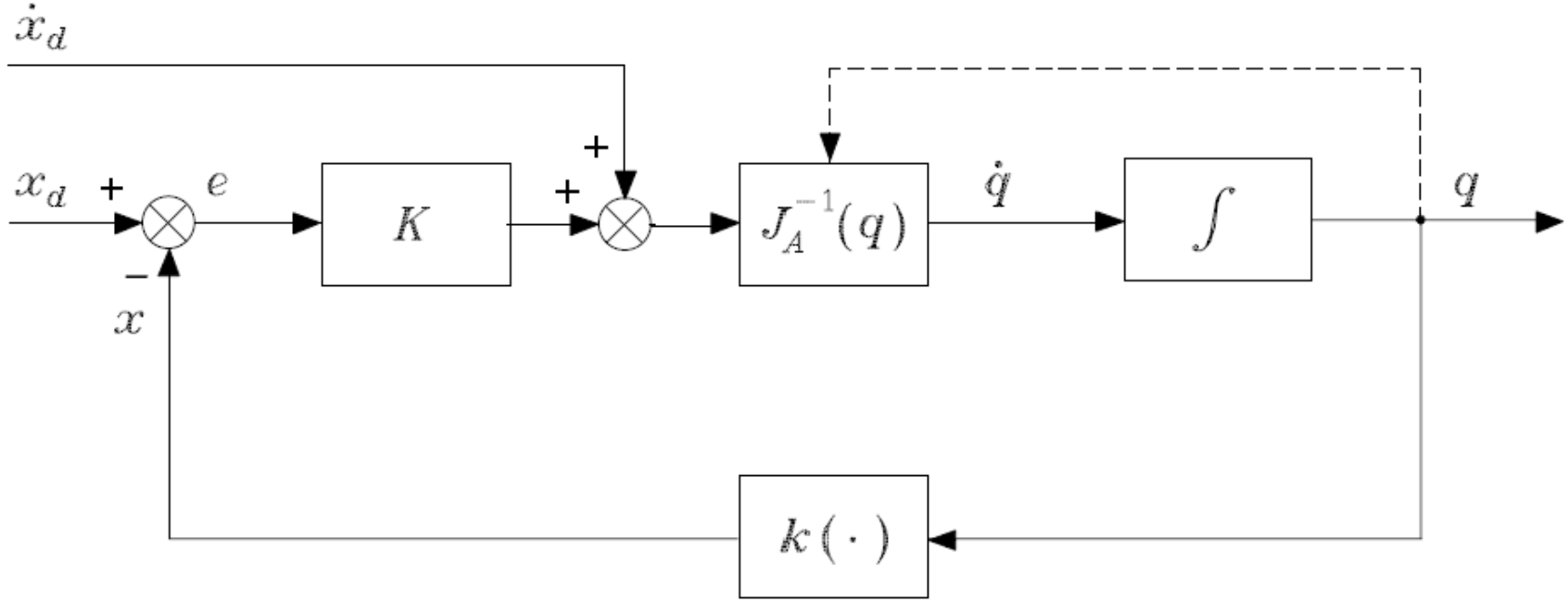
Inversa dello Jacobiano

Se adottiamo la seguente legge di dipendenza di $\dot{\boldsymbol{q}}$ da **e:**

$$\dot{\boldsymbol{q}} = \boldsymbol{J_A^{-1}(q)(\dot{x}_d + Ke)}$$

Otteniamo: $\dot{\boldsymbol{e}} + \boldsymbol{Ke} = \boldsymbol{0}$

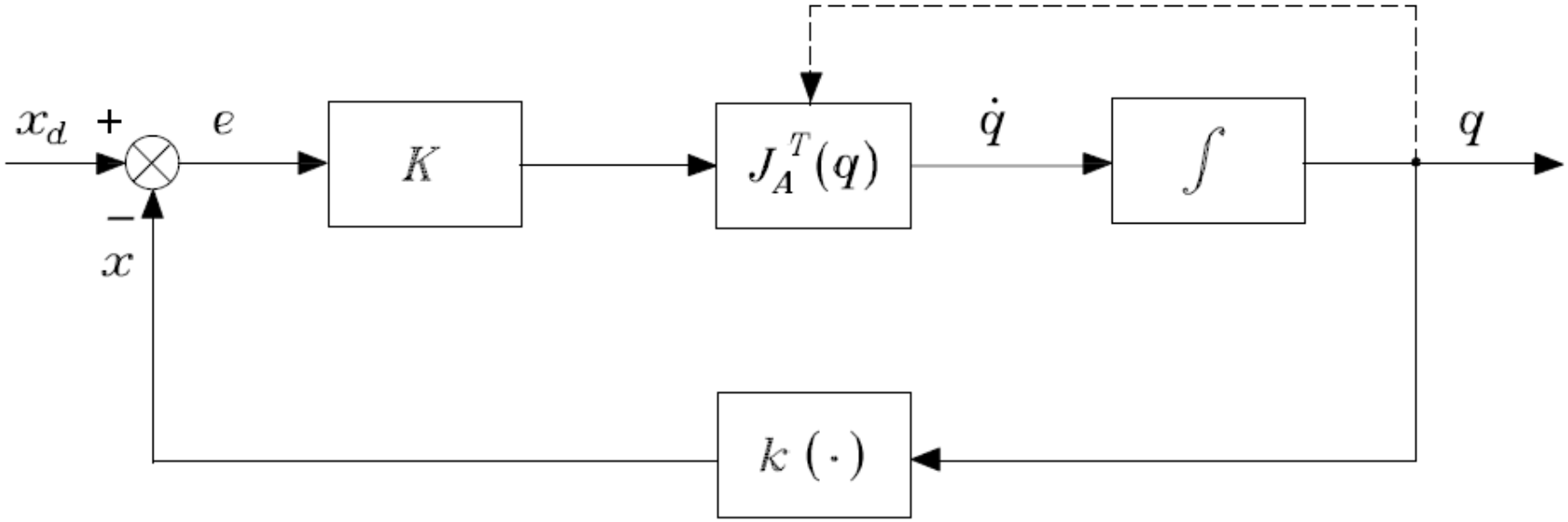
Dove $\boldsymbol{e} = \boldsymbol{x_d - x}$



Trasposta dello Jacobiano

Se adottiamo la seguente legge di dipendenza di $\dot{\boldsymbol{q}}$ da **e:**

$$\dot{\boldsymbol{q}} = \boldsymbol{J}_A^{\boldsymbol{T}}(\boldsymbol{q})\boldsymbol{K}\boldsymbol{e}$$

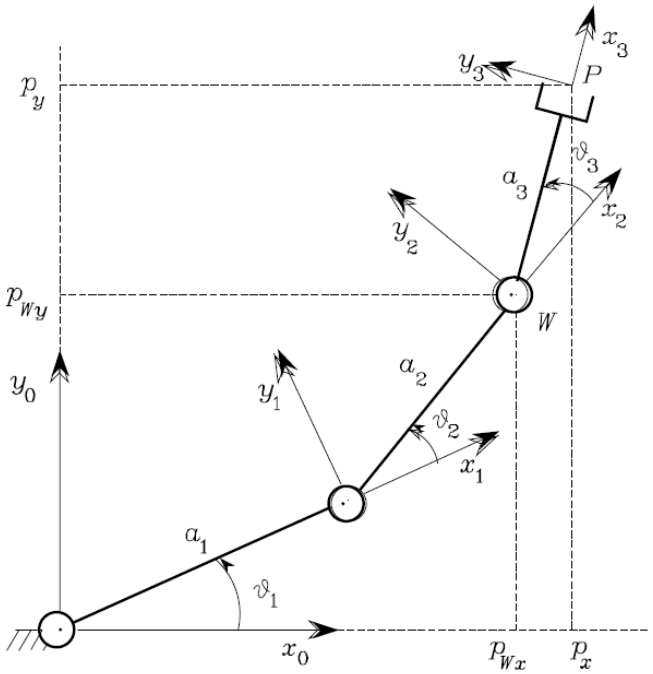


Jacobiano geometrico

Mette in relazione le velocità articolari $\dot{\theta}$ con le velocità dell'end-effector nello spazio cartesiano:

$$\begin{pmatrix} v \\ \omega \end{pmatrix} = J(\theta)\dot{\theta}$$

$v \in R^3$: velocità lineare dell'end-effector
$\omega \in R^3$: velocità angolare dell'end-effector
$\dot{\theta} \in R^n$: velocità dei giunti (n 0 numero dei giunti)
$J \in R^{6xn}$: Jacobiano geometrico



$$\begin{bmatrix} \boldsymbol{J}_{Pi} \\ \boldsymbol{J}_{Oi} \end{bmatrix} = \begin{cases} \begin{bmatrix} \boldsymbol{z}_{i-1} \\ \boldsymbol{0} \end{bmatrix} & \text{per un giunto } \textit{prismatico} \\ \begin{bmatrix} \boldsymbol{z}_{i-1} \times (\boldsymbol{p} - \boldsymbol{p}_{i-1}) \\ \boldsymbol{z}_{i-1} \end{bmatrix} & \text{per un giunto } \textit{rotoidale} \end{cases}$$

z_{i-1}: asse del giunto i (nel frame i-1)
p_{i-1}: origine del frame i-1 (posizione del giunto i)
p: posizione end-effector

$$\boldsymbol{J}(\boldsymbol{q}) = \begin{bmatrix} \boldsymbol{z}_0 \times (\boldsymbol{p} - \boldsymbol{p}_0) & \boldsymbol{z}_1 \times (\boldsymbol{p} - \boldsymbol{p}_1) & \boldsymbol{z}_2 \times (\boldsymbol{p} - \boldsymbol{p}_2) \\ \boldsymbol{z}_0 & \boldsymbol{z}_1 & \boldsymbol{z}_2 \end{bmatrix}$$

Generazione di una funzione **q**(t) che interpola i valori assegnati per le variabili di giunto nel rispetto dei vincoli imposti

Moto punto-punto

Generazione di **q**(t) che realizza il moto da una configurazione $\boldsymbol{q_i}$ a $\boldsymbol{q_f}$ in un tempo assegnato t_f



Per ogni variabile di giunto q viene calcolato un polinomio di grado 3

Tramite 4 equazioni di vincolo si risolve il sistema di equazioni:

$$\begin{cases} q(t) = a_3 t^3 + a_2 t^2 + a_1 t + a_0 \\ \dot{q}(t) = 3a_3 t^2 + 2a_2 t + a_1 \\ \ddot{q}(t) = 6a_3 t + 2a_2 \end{cases}$$

$$\begin{cases} t = 0, & q_i = a_0 \\ t = 0, & \dot{q}_i = a_1 \\ t = t_f, & q_f = a_3 t_f^3 + a_2 t_f^2 + a_1 t_f + a_0 \\ t = t_f, & \dot{q}_f = 3a_3 t_f^2 + 2a_2 t_f + a_1 \end{cases}$$

Traiettorie nello SG: segmento nello spazio

Per ogni variabile di giunto q viene calcolato un polinomio di grado 3

Tramite 4 equazioni di vincolo si risolve il sistema di equazioni:

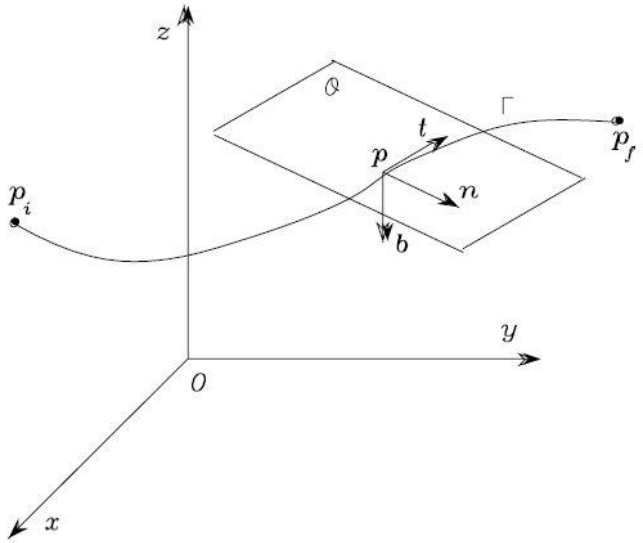
$$\begin{cases} t = 0, & q_i = a_0 \\ t = 0, & \dot{q}_i = a_1 \\ t = t_f, & q_f = a_3 t_f^3 + a_2 t_f^2 + a_1 t_f + a_0 \\ t = t_f, & \dot{q}_f = 3a_3 t_f^2 + 2a_2 t_f + a_1 \end{cases}$$

$$Ax = b$$

$$A = \begin{bmatrix} t_i^3 & t_i^2 & t_i & 1 \\ t_f^3 & t_f^2 & t_f & 1 \\ 3t_i^2 & 2t_i & 1 & 0 \\ 3t_f^2 & 2t_f & 1 & 0 \end{bmatrix}$$

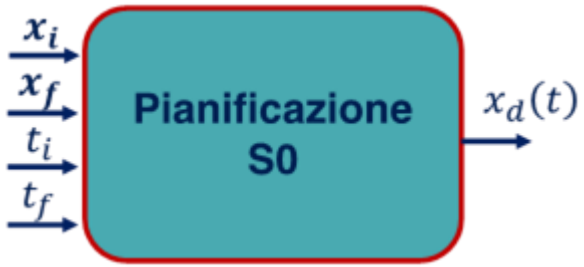
$$x = \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} \qquad b = \begin{bmatrix} q_i \\ q_f \\ \dot{q}_i \\ \dot{q}_f \end{bmatrix}$$

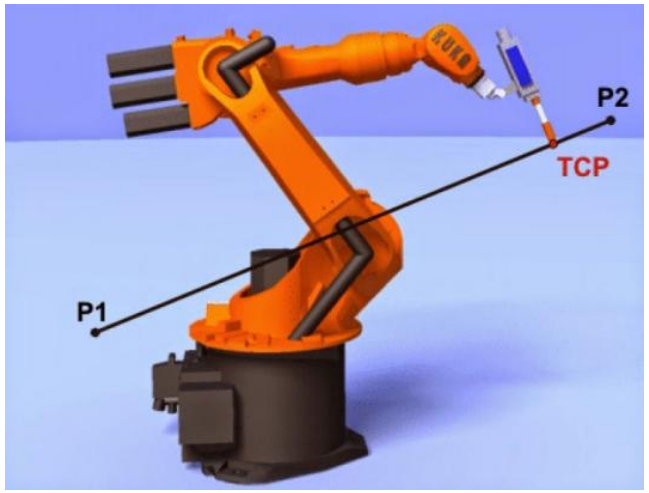
Traiettorie nello SO: segmento nello spazio

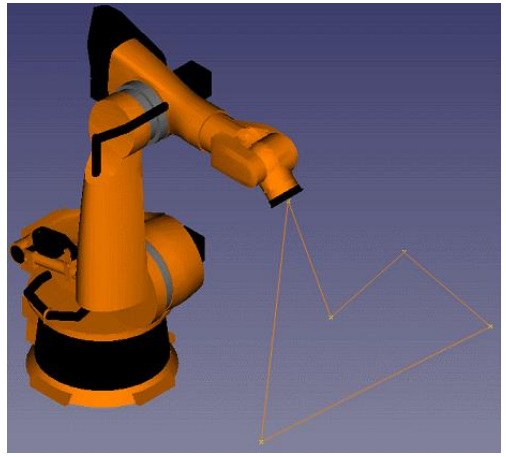


Generazione di **p**(t) che realizza il moto da una posizione iniziale $\boldsymbol{p_i}$ a $\boldsymbol{p_f}$ in un tempo assegnato

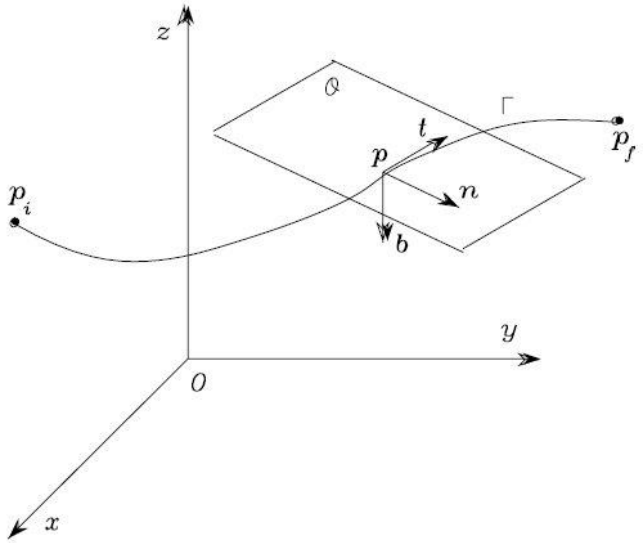
Un percorso lineare è completamente caratterizzato una volta dati due punti nello spazio cartesiano







Traiettorie nello SO: segmento nello spazio



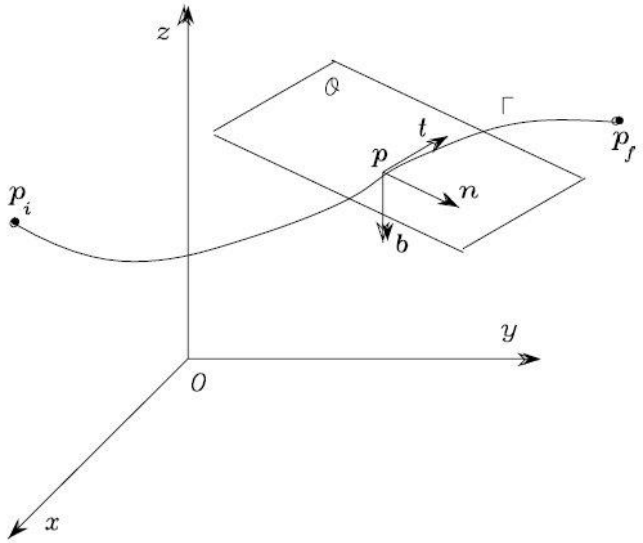
$$\boldsymbol{p} = \boldsymbol{f}(\sigma)$$

Si ottiene quindi una rappresentazione parametrica del percorso Γ rispetto al parametro scalare σ

All'aumentare di σ, il punto $\boldsymbol{p}$ si muove su Γ in una certa direzione

Definiamo quindi $\boldsymbol{p_i}$ come un punto sul percorso aperto e definiamo una direzione di percorrenza.
La lunghezza dell'arco, o **ascissa curvilinea**, s di un generico punto $\boldsymbol{p}$ rappresenta la lunghezza dell'arco di Γ con estremi $\boldsymbol{p}$ e $\boldsymbol{p_i}$

Traiettorie nello SO: segmento nello spazio



$$\boldsymbol{p} = \boldsymbol{f}(s)$$

Ad ogni valore di s corrisponde quindi un punto sul percorso Γ

Definendo quindi una **legge oraria** per s, $s(t)$, siamo in grado di generare il riferimento in posizione $\boldsymbol{p}(t) = \boldsymbol{p}(s(t))$ che stiamo cercando per ogni istante di tempo

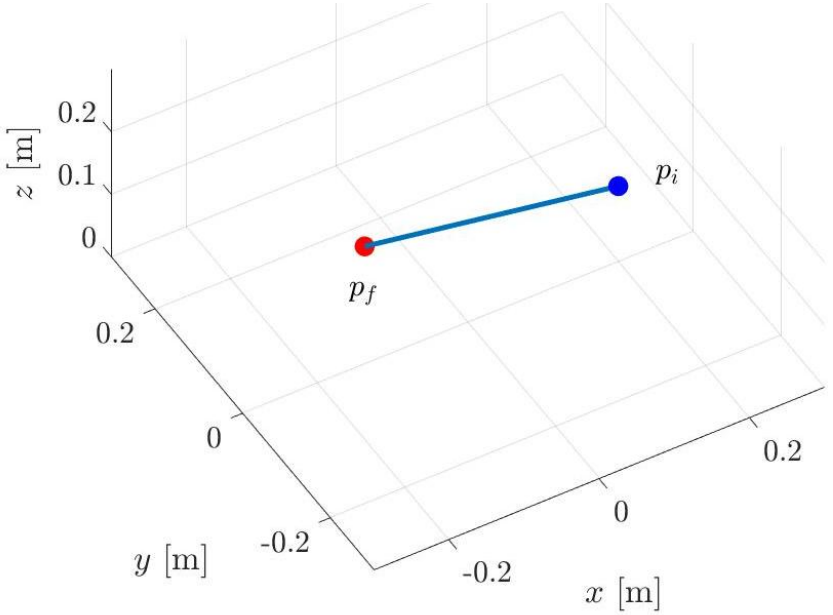
Traiettorie nello SO: segmento nello spazio

$$\boldsymbol{p}(s) = \boldsymbol{p_i} + \frac{s(t)}{\|\boldsymbol{p_f} - \boldsymbol{p_i}\|}(\boldsymbol{p_f} - \boldsymbol{p_i})$$

Definizione della legge oraria per $s(t)$

$$s(t) = a_5 t^5 + a_4 t^4 + a_3 t^3 + a_2 t^2 + a_1 t + a_0$$

Quali condizioni dobbiamo imporre?



Traiettorie nello SO: segmento nello spazio

$$s(t) = a_5 t^5 + a_4 t^4 + a_3 t^3 + a_2 t^2 + a_1 t + a_0$$

6 parametri liberi

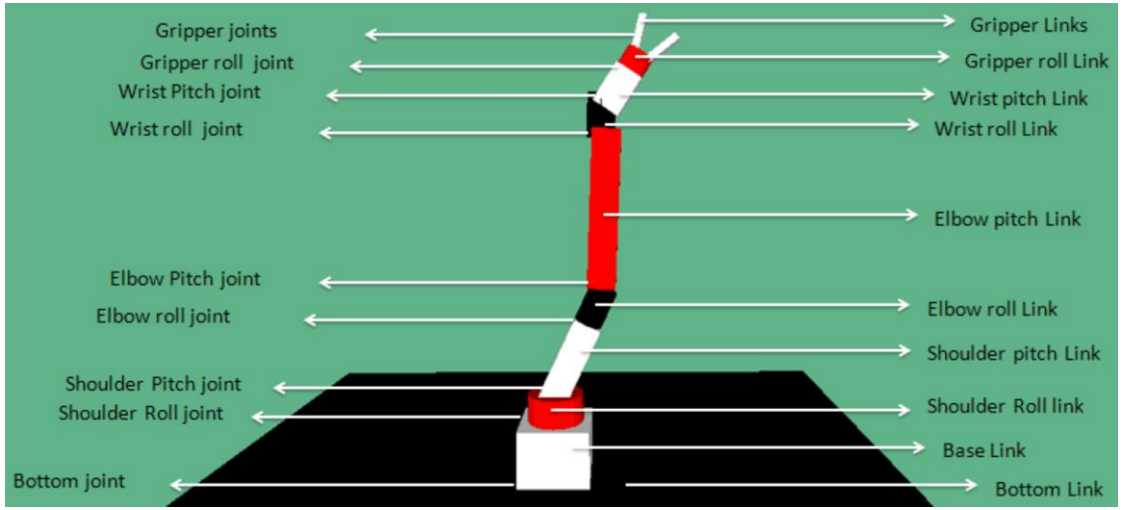
6 condizioni di vincolo

s $\dot{s}$ $\ddot{s}$

$$\begin{cases} t = 0, & 0 = a_0 \\ t = 0, & 0 = a_1 \\ t = 0, & 0 = a_2 \\ t = t_f, & \|p_f - p_i\| = a_5 t_f^5 + a_4 t_f^4 + a_3 t_f^3 + a_2 t_f^2 + a_1 t_f + a_0 \\ t = t_f, & 0 = 5a_5 t_f^4 + 4a_4 t_f^3 + 3a_3 t_f^2 + 2a_2 t_f + a_1 \\ t = t_f, & 0 = 20a_5 t_f^3 + 12a_4 t_f^2 + 6a_3 t_f + 2a_2 \end{cases}$$

Invertire la cinematica del manipolatore

1. Pianificazione del movimento da una posizione iniziale a finale
2. Calcolo della traiettoria
3. Visualizzazione del movimento in tempo reale



DH

d	θ (°)	a	α (°)
0.08	0	0	90
0	0	0	-90
0.17	0	0	-90
0	0	0	90
0.25	0	0	90
0	0	0	-90
0.04	0	0	0

Joint number	Joint name	Joint type	Angle limits (in degrees)
1	bottom_joint	Fixed	--
2	shoulder_pan_joint	Revolute	-150 to 114
3	shoulder_pitch_joint	Revolute	-67 to 109
4	elbow_roll_joint	Revolute	-150 to 41
5	elbow_pitch_joint	Revolute	-92 to 110
6	wrist_roll_joint	Revolute	-150 to 150
7	wrist_pitch_joint	Revolute	-92 to 113
8	gripper_roll_joint	Revolute	-150 to 150

Python

- Usare la funzione dh_to_transformation_matrix(d,theta,a,alpha)
- Implementare la funzione forward_kinematics(dh_params, joint_angles) per il calcolo della cinematica diretta
- Implementare la funzione calculate_geometric_jacobian(dh_params, joint_angles) → calcolo del Jacobiano J per trasformare velocità articolari in velocità cartesiane
- Implementare la funzione cartesianPlanner(pi, pf, ti, tf, t) → pianifica una traiettoria lineare planare tra 2 punti nello spazio 3D
- Implementare la funzione inverse_kinematics_jacobian(dh_params, joint_angles, target_position, joint_limits, max_iterations=1000, tol=1e-2)→ risolve IK usando il metodo della pseudo-inversa del Jacobiano